

```

import heapq

class PuzzleState:
    def __init__(self, board, parent, move, depth, cost):
        self.board = board
        self.parent = parent
        self.move = move
        self.depth = depth
        self.cost = cost

    def __lt__(self, other):
        return self.cost < other.cost

def manhattan_distance(board, goal):
    distance = 0
    for i in range(1, 9):
        x1, y1 = divmod(board.index(i), 3)
        x2, y2 = divmod(goal.index(i), 3)
        distance += abs(x1 - x2) + abs(y1 - y2)
    return distance

def a_star_search(initial_state, goal_state):
    explored = set()
    priority_queue = []
    initial = PuzzleState(initial_state, None, None, 0, manhattan_distance(initial_state, goal_state))
    heapq.heappush(priority_queue, initial)

    while priority_queue:
        current_state = heapq.heappop(priority_queue)
        explored.add(tuple(current_state.board))

        if current_state.board == goal_state:
            return current_state

        for move in possible_moves(current_state.board):
            new_board = apply_move(current_state.board, move)
            if tuple(new_board) not in explored:
                new_state = PuzzleState(new_board, current_state, move, current_state.depth + 1,
current_state.depth + 1 + manhattan_distance(new_board, goal_state))
                heapq.heappush(priority_queue, new_state)

    return None

```

```

def possible_moves(board):
    moves = []
    empty_index = board.index(0)
    if empty_index % 3 > 0: # Move left
        moves.append(-1)
    if empty_index % 3 < 2: # Move right
        moves.append(1)
    if empty_index > 2: # Move up
        moves.append(-3)
    if empty_index < 6: # Move down
        moves.append(3)
    return moves

def apply_move(board, move):
    new_board = board[:]
    empty_index = new_board.index(0)
    new_index = empty_index + move
    new_board[empty_index], new_board[new_index] = new_board[new_index],
new_board[empty_index]
    return new_board

# Example usage
initial = [1, 2, 3, 4, 5, 6, 0, 7, 8]
goal = [1, 2, 3, 4, 5, 6, 7, 8, 0]
result = a_star_search(initial, goal)
if result:
    print("Puzzle solved!")
else:
    print("No solution found.")

```